گزارش پیاده‌سازی سرور تراکنش‌های بانکی توزیع‌شده

سینا اخوی

github.com/Sina-Akhavi/distributed-systems-course

مهر ۱۴۰۵

فهرست مطالب

۱ مقدمه ۳

۲ طراحی معماری و روش ارتباطی **(Communication Method)** ۳

۱.۲ دلیل انتخاب پروتکل TCP ۳

۲.۲ ساختار پیام‌ها (Message Payload) ۳

۳ مدیریت کلاینت‌های همزمان **(Concurrency Management)** ۴

۱.۳ معماری چندنخی (Multi-threading) ۴

۲.۳ محدودسازی اتصالات با سمافور (Semaphore) ۴

۴ مدیریت وضعیت مشترک و صحت عملیات **(Shared State & Correctness)** ۴

۱.۴ جلوگیری از گم شدن به‌روزرسانی‌ها (Lost Updates) ۴

۵ تضمین ویژگی‌های **ACID** ۵

۶ ارزیابی و سناریوهای تست ۶

۱.۶ راه‌اندازی سرور با محدودیت کلاینت‌ها ۶

۲.۶ تست عملیات موفق (Happy Path) ۶

۳.۶ تست مدیریت استثناها (Exception Handling) ۷

۴.۶ تست همزمانی با اجرای حلقه For ۸

۷ دسترسی به کد منبع ۸

۸ نتیجه‌گیری ۸

۱ مقدمه

امروزه سیستم‌های توزیع‌شده (Distributed Systems) نقش حیاتی در زیرساخت‌های نرم‌افزاری ایفا می‌کنند. یکی از کلاسیک‌ترین و در عین حال چالش‌برانگیزترین کاربردهای این سیستم‌ها، پیاده‌سازی یک سیستم بانکی است که در آن چندین کلاینت به صورت همزمان سعی در خواندن یا تغییر موجودی حساب‌ها دارند. هدف از این پروژه، طراحی و پیاده‌سازی یک معماری کلاینت-سرور (Client-Server) با قابلیت پردازش همزمان، حفظ یکپارچگی داده‌ها و رعایت اصول تراکنش‌های پایگاه داده است. این گزارش به بررسی معماری سیستم، روش‌های مدیریت همزمانی، تضمین ویژگی‌های ACID و ارزیابی عملی سیستم می‌پردازد.

۲ طراحی معماری و روش ارتباطی (Communication Method)

برای برقراری ارتباط بین ماشین‌های مجازی کلاینت و سرور، از برنامه‌نویسی سوکت (Socket Programming) بر بستر پروتکل TCP در لایه انتقال استفاده شده است.

۱.۲ دلیل انتخاب پروتکل TCP

در سیستم‌های مالی، از دست رفتن حتی یک بسته اطلاعاتی (Packet Loss) می‌تواند به فاجعه منجر شود. پروتکل TCP به دلیل اتصال‌گرا بودن (Connection-oriented)، تضمین تحویل داده‌ها، و کنترل جریان (Flow Control) انتخاب ایده‌آلی برای این معماری است. سیستم تضمین می‌کند که درخواست‌های برداشت یا واریز دقیقاً به همان ترتیبی که ارسال شده‌اند، در سمت سرور دریافت و پردازش شوند.

۲.۲ ساختار پیام‌ها (Message Payload)

تمامی پیام‌های مبادله شده بین کلاینت و سرور به صورت رشته‌های کدگذاری شده در قالب استاندارد JSON ارسال و دریافت می‌شوند. دلیل این انتخاب، مستقل بودن JSON از زبان برنامه‌نویسی و ساختاریافته بودن آن است. هر درخواست شامل فیلدهایی نظیر نوع عملیات (action)، شماره حساب (account) و مبلغ تراکنش (amount) است. پاسخ‌های سرور نیز همواره شامل وضعیت موفقیت یا شکست عملیات (status) و پیام‌های خطای مربوطه می‌باشند.

۳ مدیریت کلاینت‌های همزمان (Concurrency Management)

یک سرور بانکی استاندارد باید بتواند به چندین مشتری در لحظه پاسخ دهد بدون آنکه درخواست‌های آن‌ها باعث مسدود شدن سیستم (Blocking) گردد.

۱.۳ معماری چندنخی (Multi-threading)

سرور این پروژه بر پایه معماری چندنخی طراحی شده است. نخ اصلی (Main Thread) سرور در یک حلقه بی‌نهایت منتظر اتصال کلاینت‌ها می‌ماند. با ورود هر درخواست اتصال جدید، سرور فوراً یک نخ (Thread) مجزا برای پردازش آن کلاینت ایجاد می‌کند و خود به انتظار برای دریافت کلاینت بعدی برمی‌گردد.

۲.۳ محدودسازی اتصالات با سمافور (Semaphore)

برای جلوگیری از مصرف بیش از حد منابع شبکه و پردازنده در ماشین مجازی سرور (جلوگیری از حملات DoS یا اتمام حافظه)، حداکثر تعداد کلاینت‌های همزمان نباید در کد ثابت (Hard-coded) باشد. این مقدار از طریق متغیرهای محیطی (فایل env.) یا آرگومان خط فرمان max-clients– در زمان اجرای سرور دریافت می‌شود. برای اعمال این محدودیت از مکانیزم threading.Semaphore در پایتون استفاده شده است. اگر تعداد نخ‌های فعال به سقف تعیین شده برسد، سمافور اجازه ایجاد نخ جدید را نمی‌دهد و اتصالات جدید در صف انتظار شبکه باقی می‌مانند تا یکی از کلاینت‌های فعلی کار خود را به اتمام برساند.

۴ مدیریت وضعیت مشترک و صحت عملیات (Shared State & Correctness)

وضعیت مشترک (Shared State) سیستم که شامل موجودی لحظه‌ای حساب‌هاست، درون یک فایل پایگاه داده رابطه‌ای SQLite به نام bank.db نگهداری می‌شود.

۱.۴ جلوگیری از گم شدن به‌روزرسانی‌ها (Lost Updates)

چالش اصلی در پردازش همزمان این است که اگر دو نخ به طور همزمان موجودی ۱۰۰۰ تومانی یک حساب را بخوانند و هر دو بخواهند ۱۰۰ تومان از آن برداشت کنند، ممکن است هر دو مقدار ۹۰۰ را محاسبه کرده و در

دیتابیس بنویسند. در نتیجه، به جای اینکه ۲۰۰ تومان از حساب کسر شود، تنها ۱۰۰ تومان کسر شده است.

در این پروژه، این مشکل با واگذاری مدیریت همزمانی به موتور SQLite و استفاده از دستور BEGIN IMMEDIATE حل شده است. این دستور به محض شروع تراکنش، یک قفل انحصاری (Exclusive Lock) برای نوشتن روی کل دیتابیس ایجاد می‌کند. در نتیجه، اگر ده کلاینت به طور همزمان درخواست برداشت ارسال کنند، نخ‌های سرور در صف قرار گرفته و عملیات خواندن، محاسبه و نوشتن موجودی جدید را به صورت کاملاً متوالی و ایمن انجام می‌دهند.

۵ تضمین ویژگی‌های ACID

در سیستم‌های توزیع‌شده مالی، رعایت چهار ویژگی ACID برای تراکنش‌ها الزامی است که در این پروژه به شکل زیر پیاده‌سازی شده‌اند:

- یکپارچگی **(Atomicity)**: هر تراکنش (مثلاً برداشت از حساب) به صورت یک واحد غیرقابل تقسیم در نظر گرفته می‌شود. اگر تمام بررسی‌ها (موجودی کافی، حساب معتبر) موفق باشد، تغییرات با commit قطعی می‌شوند و اگر در هر مرحله‌ای خطایی رخ دهد، کل عملیات با دستور rollback لغو شده و دیتابیس به حالت اولیه بازمی‌گردد.

- هم‌خوانی **(Consistency)**: سیستم هرگز نباید در وضعیتی نامعتبر قرار گیرد. علاوه بر اعتبارسنجی‌های سطح کد پایتون (مثل جلوگیری از واریز مبلغ منفی)، یک قید محدودکننده پایگاه داده به صورت CHECK(balance >= 0) تعریف شده است تا تحت هیچ شرایطی، موجودی حساب در سطح دیسک منفی نشود.

- انزوا **(Isolation)**: با استفاده از قفل BEGIN IMMEDIATE، بالاترین سطح انزوا (معادل Serializable) برای تراکنش‌های نوشتن اعمال شده است تا هیچ تراکنشی نتواند تغییرات در حال انجام تراکنش دیگر را مشاهده کند.

- پایداری **(Durability)**: موتور SQLite تغییرات کامیت‌شده را به صورت فیزیکی روی فایل دیسک می‌نویسد. در صورت توقف ناگهانی سرور (Crash) یا قطعی برق ماشین مجازی، اطلاعات حساب‌ها به هیچ وجه از بین نخواهند رفت.

۶ ارزیابی و سناریوهای تست

در این بخش، نتایج اجرای عملی کلاینت و سرور روی ماشین‌های مجازی مجزا همراه با مستندات تصویری ارائه شده است.

۱.۶ راه‌اندازی سرور با محدودیت کلاینت‌ها

همان‌طور که در شکل ۱ مشاهده می‌شود، سرور از طریق آرگومان‌های خط فرمان راه‌اندازی شده و سقف تعداد کلاینت‌های همزمان روی آن اعمال گردیده است.

```
sina@sina-machine:~/project$ python3 server.py --max-clients 5
[STARTING] Server is listening on 0.0.0.0:65432 (Max Clients: 5)
2026-10-15T20:06:12.689Z req=fce6 ip=192.168.56.105 action=withdraw account=A1001 result=success balance=2090.0 latency=2020ms
2026-10-15T20:06:12.725Z req=25db ip=192.168.56.105 action=withdraw account=A1001 result=success balance=2080.0 latency=2010ms
2026-10-15T20:06:12.735Z req=4194 ip=192.168.56.105 action=withdraw account=A1001 result=success balance=2070.0 latency=2018ms
2026-10-15T20:06:12.746Z req=2196 ip=192.168.56.105 action=withdraw account=A1001 result=success balance=2060.0 latency=2028ms
2026-10-15T20:06:12.762Z req=50c6 ip=192.168.56.105 action=withdraw account=A1001 result=success balance=2050.0 latency=2045ms
2026-10-15T20:06:14.700Z req=9232 ip=192.168.56.105 action=withdraw account=A1001 result=success balance=2040.0 latency=2008ms
2026-10-15T20:06:14.738Z req=e1de ip=192.168.56.105 action=withdraw account=A1001 result=success balance=2030.0 latency=2010ms
2026-10-15T20:06:14.753Z req=b84a ip=192.168.56.105 action=withdraw account=A1001 result=success balance=2020.0 latency=2016ms
2026-10-15T20:06:14.765Z req=3173 ip=192.168.56.105 action=withdraw account=A1001 result=success balance=2010.0 latency=2017ms
2026-10-15T20:06:14.777Z req=d614 ip=192.168.56.105 action=withdraw account=A1001 result=success balance=2000.0 latency=2012ms
```

شکل ۱: راه‌اندازی سرور با استفاده از سوییچ –max-clients

۲.۶ تست عملیات موفق **(Happy Path)**

کلاینت می‌تواند عملیات‌هایی نظیر استعلام موجودی، واریز و برداشت را انجام دهد. در شکل ۲، یک سناریوی موفقیت‌آمیز از ارتباط کلاینت با سرور به همراه پاسخ‌های ساختاریافته JSON که حاوی موجودی جدید است، نشان داده شده است.

```
sina@sunflower-vm:~/project$ python3 client.py balance --account A1001
Sending: {'action': 'balance', 'account': 'A1001'}
Response: {
  "status": "success",
  "account": "A1001",
  "balance": 1900
}
sina@sunflower-vm:~/project$ python3 client.py deposit --account A1001 --amount 500
Sending: {'action': 'deposit', 'account': 'A1001', 'amount': 500.0}
Response: {
  "status": "success",
  "account": "A1001",
  "balance": 2400.0
}
sina@sunflower-vm:~/project$ python3 client.py withdraw --account A1001 --amount 200
Sending: {'action': 'withdraw', 'account': 'A1001', 'amount': 200.0}
Response: {
  "status": "success",
  "account": "A1001",
  "balance": 2200.0
}
```

شکل ۲: اجرای موفقیت‌آمیز درخواست‌ها و دریافت پاسخ استاندارد

۳.۶ تست مدیریت استثناها (Exception Handling)

مقاومت سیستم در برابر ورودی‌های نامعتبر اهمیت بالایی دارد. شکل ۳ نشان می‌دهد که سرور چگونه درخواست‌های نامعتبر نظیر حساب کاربری ناموجود، درخواست برداشت مبالغ منفی، برداشت بیشتر از موجودی، و همچنین پیام‌های JSON خراب را با پیام خطای مناسب و بدون توقف سرور (Crash) مدیریت می‌کند.

```
sina@sunflower-vm:~/project$ python3 client.py withdraw --account A1001 --amount 50000
Sending: {'action': 'withdraw', 'account': 'A1001', 'amount': 50000.0}
Response: {
  "status": "error",
  "message": "Insufficient funds"
}
sina@sunflower-vm:~/project$ python3 client.py withdraw --account A1001 --amount -50
Sending: {'action': 'withdraw', 'account': 'A1001', 'amount': -50.0}
Response: {
  "status": "error",
  "message": "Negative or invalid amount"
}
sina@sunflower-vm:~/project$ python3 client.py malformed
Sending: {'action': 'withdraw'}
Response: {
  "status": "error",
  "message": "Missing fields"
}
sina@sunflower-vm:~/project$ python3 client.py broken_json
Response: {"status": "error", "message": "Broken JSON"}
```

شکل ۳: مدیریت استثناها و ورودی‌های نامعتبر از سوی سرور

۴.۶ تست همزمانی با اجرای حلقه For

برای اثبات مقاومت سیستم در برابر مشکل تداخل تراکنش‌ها (Race Condition)، یک حلقه در محیط Bash ماشین مجازی کلاینت اجرا شد که تعداد زیادی درخواست را در یک لحظه برای سرور ارسال می‌کند (اجرای کلاینت‌ها در پس‌زمینه با علامت &). مطابق شکل ۴، با وجود ارسال همزمان درخواست‌ها، به لطف قفل‌گذاری صحیح دیتابیس، موجودی نهایی دقیقاً به اندازه مجموع مبالغ برداشت شده کاهش می‌یابد و هیچ درخواستی نادیده گرفته نمی‌شود.

```
sina@sunflower-vm:~/project$ for i in {1..10}; do python3 client.py withdraw --account A1001 --amount 10 & done; wait
[1] 2074
[2] 2075
[3] 2076
[4] 2077
[5] 2078
[6] 2079
[7] 2080
[8] 2081
[9] 2082
[10] 2083
Sending: {'action': 'withdraw', 'account': 'A1001', 'amount': 10.0}
Sending: {'action': 'withdraw', 'account': 'A1001', 'amount': 10.0}
Sending: {'action': 'withdraw', 'account': 'A1001', 'amount': 10.0}
Sending: {'action': 'withdraw', 'account': 'A1001', 'amount': 10.0}
Sending: {'action': 'withdraw', 'account': 'A1001', 'amount': 10.0}
Sending: {'action': 'withdraw', 'account': 'A1001', 'amount': 10.0}
Sending: {'action': 'withdraw', 'account': 'A1001', 'amount': 10.0}
Sending: {'action': 'withdraw', 'account': 'A1001', 'amount': 10.0}
Sending: {'action': 'withdraw', 'account': 'A1001', 'amount': 10.0}
Sending: {'action': 'withdraw', 'account': 'A1001', 'amount': 10.0}
```

شکل ۴: اجرای همزمان چندین کلاینت از طریق حلقه For جهت تست فشار سیستم

۷ دسترسی به کد منبع

سورس‌کد کامل این پروژه شامل فایل‌های راه‌اندازی سرور، کلاینت و مستندات اجرایی، در مخزن گیت‌هاب زیر در دسترس قرار گرفته است:

`https://github.com/Sina-Akhavi/distributed-systems-course`

۸ نتیجه‌گیری

در این پروژه، یک سیستم ساده اما قدرتمند بانکی پیاده‌سازی شد. با استفاده از سوکت‌های TCP ارتباط شبکه‌ای مطمئن بین دو ماشین مجازی برقرار گردید. همچنین استفاده از مکانیزم‌های کنترل همزمانی در پایتون (Semaphore) و امکانات تضمین یکپارچگی داده در SQLite، باعث شد تا سیستم در برابر بارهای

کاری سنگین و درخواست‌های همزمان مقاوم بوده و بتواند بدون از دست دادن اطلاعات یا ایجاد مغایرت مالی، تراکنش‌ها را مدیریت کند.